g2(τ) Emitter Classification Pipeline

Code accompanying the paper "*Physics-Based versus Data-Driven Classification of Single-Photon Quantum Emitters from Sparse Autocorrelation Data*." Benchmarks three classification approaches—Levenberg–Marquardt fitting, sequential Bayesian inference, and a feedforward neural network—for distinguishing single-photon emitters (N=1) from multi-emitter sources (N≥2) from Hanbury Brown–Twiss $g^2(\tau)$ autocorrelation data, with particular attention to performance under sparse (short-integration-time) conditions.

What's in this repository

Only the **real experimental data** is included here. Simulated training/test data is not, for space reasons—anyone wishing to reproduce or extend the analysis needs to generate it themselves from the provided real data, using the three Step_ scripts below. This is by design: it also means anyone re-running the pipeline is using the exact same generation code we did, not a static snapshot of one particular run.

```
├── Experimental_Data_Raw/                    <- provided: real HBT measurements
├── Step_00_Correct_and_relabel_experimental_data.py
├── Step_01_Extract_Params_from_experimental_data.py
├── Step_02_Generate_g2_traces_with_exp_params.py
├── LM_analysis_Simulation_data_iterspecific_smoothed.py
├── Bayesian_analysis_simulation_data_iterspecific.py
├── Encoder_analysis_simulation_data_smoothed.py
├── run_seed_sweep.py
├── summarize_seed_sweep.py
├── make_table1_barplot.py
└── README.md
```

Running the pipeline (see below) will additionally create:

```
├── Experimental_Data/        <- created by Step_00
├── experimental_calibration.csv, calibration_summary.csv  <- created by Step_01
├── Simulation_Results/       <- created by Step_02, read by all three algorithms
```

├── seed_sweep_results/ <- created by run_seed_sweep.py (optional split-stability analysis)

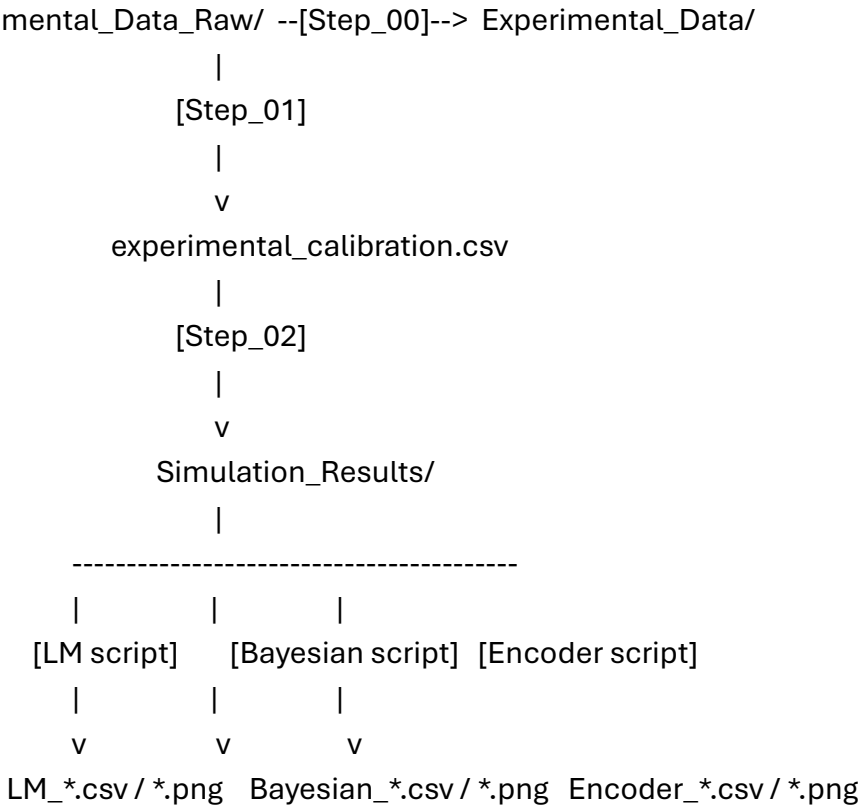
Requirements

Python 3.9+ with:
numpy
pandas
scipy
matplotlib
seaborn
scikit-learn

No GPU or deep learning framework is required—the neural-network classifier uses scikit-learn's MLPClassifier, not PyTorch/TensorFlow.

Pipeline overview

The six scripts run in a strict sequence: each one reads the previous step's output. There is no orchestration script — run them one at a time from the command line (python Step_00_...py, etc.), in the same working directory, in this order:



Experimental_Data_Raw/ — real data format (provided)
Each file is one HBT measurement, named <anything>rep_<ID>_n<N>.csv or <anything>rep_<ID>_n<N>_raw.csv (the _raw suffix and any prefix before rep_ are both optional — Step_00 matches on the rep_<ID>_n<N> part). <N> is the emitter number as originally labeled during acquisition; Step_00 re-derives and may correct this (see below).

File structure (CSV, no index column):

- **Row 1**: APD1, followed by one count-rate value (Hz) per iteration column — the single-channel count rate on detector 1 at each integration time.
- **Row 2**: APD2, same for detector 2.
- **Row 3** (header row): tau, iter_1, iter_2, ..., iter_9.
- **Remaining rows**: tau (ns) plus one column of **raw, unnormalized** coincidence counts per iteration — each iter_k column is the *cumulative* histogram after that many integration steps (i.e. iter_9 contains all counts accumulated up to and including the longest integration time, not just that step's counts in isolation).

The integration time (seconds) corresponding to each iter_k column is fixed by acquisition schedule, not stored in the file itself — it's hardcoded in Step_00 and Step_01 as [1, 2, 5, 10, 20, 50, 100, 200, 500] for iter_1…iter_9. **If your acquisition used a different schedule, update ITERATION_INTEGRATION_TIMES_S in both Step_00 and Step_01 before running anything.**

Optionally, a file ending in _norm.csv (otherwise identically named) can accompany a _raw.csv file — the instrument's own normalized $g^2(\tau)$ output for the same measurement. If present, Step_00 uses it to derive an exact baseline; if absent, Step_00 falls back to computing the baseline from the APD rates directly (accurate to within 1-3%, validated against files where both are available).

Step_00 — Step_00_Correct_and_relabel_experimental_data.py
Reads: Experimental_Data_Raw/ **Writes**: Experimental_Data/
Corrects a small, fixed timing offset between the two detector channels (typically a few nanoseconds to tens of nanoseconds, detected per file and re-centered so the antibunching dip sits at $\tau=0$), and independently re-derives each file's true emitter number from a classical amplitude threshold on the corrected $g^2(0)$ — since a file's original N label (from the filename) may not survive timing correction. Files whose corrected $g^2(0)$ falls too close to a classification threshold are excluded as genuinely ambiguous rather than forced into a class.

For each surviving file, writes two outputs to Experimental_Data/:

- rep_<ID>_n<N>.csv — same tau/iter_ column format as the input, but corrected and properly normalized to $g^2(\tau)$.
- rep_<ID>_n<N>_rawcounts.csv — a companion file with the same corrected data as raw (unnormalized) counts, used downstream for proper Poisson-weighted fitting.

Prints a full audit log (detected shift, both $g^2(0)$ estimation methods, and the reason for every exclusion) to the console — worth reading through once to sanity-check your own data before proceeding.

Run: python Step_00_Correct_and_relabel_experimental_data.py

Step_01 — Step_01_Extract_Params_from_experimental_data.py
Reads: Experimental_Data/ (Step_00's output) **Writes**: experimental_calibration.csv, calibration_summary.csv
Extracts, from each corrected real file, the background fraction f_bg (and bunching pedestal A2) needed to calibrate realistic noise levels in the synthetic data generated by Step_02. f_bg = B/(S+B), recovered by inverting the emitter-dilution relation $g^2(0) = 1 - \rho^2/N$ for the file's known N and measured $g^2(0)$ — not measured directly.

Results are pooled by emitter number: N=1 on its own, and N=2 through 5 pooled together (individually there are usually too few real multi-emitter files to characterize each N separately). If even the pooled N≥2 group is too small to be confident, N=1 data is blended in as a fallback — this and every other pooling decision is printed to the console.

Run: python Step_01_Extract_Params_from_experimental_data.py

Step_02 — Step_02_Generate_g2_traces_with_exp_params.py
Reads: experimental_calibration.csv (Step_01's output) **Writes**: Simulation_Results/
Generates the large synthetic dataset the three classifiers are actually trained and evaluated on — a Gillespie Monte Carlo simulation of a 3-level emitter, with the background fraction for each simulated file bootstrapped from the real calibration pool (rather than an idealized constant), so simulated noise statistics match what was actually measured.

Key parameters, all set near the top of the if __name__ == "__main__":-equivalent section of the script (search for TWEAKABLE GENERATION PARAMETERS):

Parameter	Meaning
repetitions	Number of simulated files to generate (5000 in the paper)
max_emitters	Emitter number is drawn uniformly from 1 to this value (5 in the paper — gives ~20% Single / ~80% Multi)
iterations	Number of accumulated integration-time snapshots per file (300 in the paper)
F_BG_OVERRIDE	Set to a number instead of None to force every file to the same background fraction, bypassing the real-data bootstrap (useful for controlled experiments)

This step is the most expensive one to run — expect a runtime of several hours for 5000 repetitions at 300 iterations each, depending on hardware. Consider reducing repetitions for a first test run.

Outputs, one pair per repetition: g2_raw_rep_<ID>_n<N>.csv (thinned raw counts) and g2_norm_rep_<ID>_n<N>.csv (normalized $g^2(\tau)$), plus a single training_labels.csv recording every repetition's ground-truth emitter number, injected background fraction, and a self-consistency check (background fraction and $g^2(0)$ re-extracted from the simulation's own output, for comparison against what was injected).

Run: python Step_02_Generate_g2_traces_with_exp_params.py

The three classifiers

All three read from Simulation_Results/ (Step_02's output) and share:

- The same binary task: N=1 ("Single") vs. N≥2 ("Multi").
- The same 5-fold stratified cross-validation (n_folds=5, random_state=42) — every file is used as a held-out test file exactly once, by a model/threshold that never saw it during calibration or training.
- The same set of 30 evaluated integration-time points (iteration_stride=10, i.e. iterations 10, 20, ..., up to iterations set in Step_02).
- The same three output types: a metrics-vs-iteration table/plot (Recall, Precision, Leakage — see the paper's Methods section for definitions), a confusion matrix at the final iteration, and a per-file correctness heatmap across all iterations — so all three methods' outputs are directly comparable.

No settings need to change between scripts for a first run; each script's SETTINGS dictionary near the top can be edited if you want to point at a different Simulation_Results folder, change the number of folds, or adjust the smoothing strength (ema_alpha, LM and Encoder only).

Script	**Method**	**Key outputs (Simulation_Results/)**
LM_analysis_Simulation_data_iterspecific_smoothed.py	Levenberg–Marquardt, per-iteration threshold calibration + smoothing	LM_metrics_vs_iteration.csv/.png, LM_predictions_long.csv, LM_confusion_matrix_last_iter.png, LM_correctness_heatmap.png
Bayesian_analysis_simulation_data_iterspecific.py	Sequential Bayesian inference, per-chain-position likelihood calibration	Bayesian_metrics_vs_iteration.csv/.png, Bayesian_predictions_long.csv, Bayesian_confusion_matrix_last_iter.png, Bayesian_correctness_heatmap.png
Encoder_analysis_simulation_data_smoothed.py	Feedforward neural network (scikit-learn MLPClassifier) + smoothing	Encoder_metrics_vs_iteration.csv/.png, Encoder_predictions_long.csv, Encoder_confusion_matrix_last_iter.png, Encoder_correctness_heatmap.png

Each can be run independently and in any order, once Simulation_Results/ exists:

python LM_analysis_Simulation_data_iterspecific_smoothed.py

python Bayesian_analysis_simulation_data_iterspecific.py

python Encoder_analysis_simulation_data_smoothed.py

Runtime note: the LM script recalibrates its decision threshold separately at every evaluated iteration, which costs substantially more compute than a single fixed-threshold calibration — expect it to be the slowest of the three on a full 5000-file run.

Split-stability analysis and summary figure (optional)

These three scripts are not part of the main pipeline. They use the outputs of the pipeline above to check how much the reported results depend on one particular random train/test split, and to present the headline comparison as a figure. All three expect to sit in the same folder as the three classifier scripts, with Simulation_Results/ inside it, and need no arguments.

run_seed_sweep.py

Reads: Simulation_Results/ and the three classifier scripts above. **Writes**: seed_sweep_results/seed_<N>/{LM,Bayesian,MLP}_predictions_long.csv

Repeats the full 5-fold stratified cross-validation of all three classifiers under several different random seeds, i.e. several different assignments of files to folds. The classifier scripts themselves are not modified: each one takes all of its randomness from SETTINGS["random_state"] (the fold assignment for all three; for the MLP also the oversampling and network initialisation), so this script imports each one, overrides that single setting, runs the script's own cross-validation function, applies the script's own smoothing (LM and MLP only, as in the original scripts), and writes predictions in the same format as the original *_predictions_long.csv files.

The default seeds are 42 and 101–109 (edit DEFAULT_SEEDS near the top of the script to change them); each method for each seed is written only when it has fully finished, and finished runs are skipped, so an interrupted run can simply be restarted. Seed 42 is the seed used for the main analysis, so its output should reproduce the main results exactly. Options (all optional): --seeds, --methods {LM,Bayesian,MLP}, --scripts-dir, --sim-dir, --out-dir.

Runtime note: this repeats the full classifier runs once per seed, so it is expensive. On a 5000-file dataset expect on the order of hours per seed, with the LM classifier the slowest (it recalibrates a threshold at every evaluated iteration). Run a single seed first (python run_seed_sweep.py --seeds 42) to gauge the time on your hardware.

Run: python run_seed_sweep.py

summarize_seed_sweep.py

Reads: seed_sweep_results/ (run_seed_sweep.py's output; only seeds for which all three methods have finished are used) **Writes**: into seed_sweep_results/: seed_sweep_per_seed.csv, seed_sweep_summary.csv, seed_sweep_rankings.csv, seed_sweep_figure.png

For each seed it recomputes Recall, Precision and Leakage at iterations 10 and 300 from the pooled out-of-fold predictions (as in the paper's Table 1), and the majority-vote ensemble at iteration 300 (as in Table 4). It then reports, across seeds, the mean, standard deviation, minimum and maximum of each metric, and for how many seeds each method ranking discussed in the paper holds (for example "MLP has the lowest leakage at iteration 10"). The figure shows the per-seed values with their mean. It can be run at any point during a sweep, on whichever seeds are complete. With few seeds, the min–max range and the count of seeds in which a ranking holds are more meaningful than the standard deviation.

Run: python summarize_seed_sweep.py

make_table1_barplot.py
Reads: LM_predictions_long.csv, Bayesian_predictions_long.csv, Encoder_predictions_long.csv (written by the three classifier scripts; by default looked up in the current directory, or pass --data-dir) **Writes**: Figure_Table1_barplot.png, Figure_Table1_barplot_caption.txt, table1_with_CI.csv

Draws a two-panel bar plot of Recall, Precision and Leakage for the three classifiers at the sparsest (iteration 10) and most-converged (iteration 300) integration times, with 95% Wilson score confidence intervals as error bars. The Wilson interval is used rather than the normal approximation because it remains well defined when a proportion is exactly 0 or 1, which happens routinely for Leakage; the intervals are therefore not symmetric about the point estimate near 0 or 1. At iteration 300 the Leakage values are shown in a small log-scale inset, using the same zero-value floor convention as the Leakage panel of the main metrics-vs-iteration figure. table1_with_CI.csv contains the plotted values together with their confidence bounds and the underlying TP/FN/FP/TN counts. Use --no-leakage-log-inset to disable the inset.

Run: python make_table1_barplot.py

Reproducibility notes

- All three classifiers use random_state=42 for their cross-validation split; results should be exactly reproducible given the same Simulation_Results/ data.
- Changing random_state changes which files land in which fold, and (for the MLP) the oversampling and network initialisation. run_seed_sweep.py automates repeating the analysis with other seeds; its seed-42 run should match the main results exactly. The simulated dataset itself is held fixed across seeds.

- Step_02's Monte Carlo generation is *not* seeded, so re-running it produces a different (though statistically similar) synthetic dataset each time. If you need a byte-identical dataset across runs, add an explicit np.random.seed(...) near the top of Step_02 before generation begins.
- Every script prints a full log of what it did (files excluded, folds calibrated, parameters used) to the console — worth capturing (python Step_00_....py | tee step00_log.txt) alongside your results for later reference.